Generic Triple-Latent Compression with Gated Associative Retrieval

Liu Xiao
liu.xiao.in@gmail.com

GPT-5.4 xhigh

Abstract

Can compressed latent state recover higher-order token interactions without explicitly enumerating token triplets or relying on benchmark-specific parsing? We study a small family of **generic triple-latent** sequence models that maintain a running token state together with a compressed pair-memory pathway. The resulting base architecture is recurrent, untyped, and trained only from raw token sequences. We first evaluate three variants—dense pair memory (`triple-latent`), slot-compressed pair memory (`triple-slot`), and pair memory plus local convolution (`triple-hybrid`)—against a standard Transformer baseline on byte-level WikiText-2, associative recall, and throughput scaling. In a matched-width setting, all three latent variants improve validation bits-per-byte over the Transformer baseline, with `triple-hybrid` reaching 4.766 versus 5.124. In an approximately parameter-matched language-model follow-up, `triple-hybrid` still improves on the Transformer, reaching 5.067 at 169,424 parameters versus 5.124 at 174,848 parameters. A targeted near-budget sweep finds an even stronger winner: `triple-hybrid` with $d = 64$ and state dimension 8 reaches 4.827 at 177,752 parameters, and averages 4.806 across three seeds versus 5.143 for the Transformer baseline. The latent-only models do not improve retrieval-heavy associative recall, so we add a recall-focused hybrid: separate key-value memory, previous-token queries, last-layer retrieval, and a gated late-fusion path. This gated hybrid reaches 41.9% mean associative recall across three seeds at $d = 64$, compared with 25.0% for the Transformer and 11.6% for plain `triple-hybrid`, and attains 100.0% in its best seed. The gain is seed-sensitive and the model remains far slower than attention in the current Python reference implementation. A later warmup-plus-curriculum retraining attempt collapses the gated model back to 12.7% mean recall. We interpret these results as evidence that latent compression helps generic language modeling, while exact retrieval benefits from a separate gated memory path rather than a single compressed state alone.

1 Introduction

Higher-order token interactions sit at the heart of sequence modeling, but the standard way to capture them—full self-attention—comes with quadratic sequence interactions and a strong dependence on highly optimized kernels [Vaswani et al., 2017]. Recurrent and linear-time alternatives aim to replace that cost with compressed state, but often lose expressive power in the process.

This paper studies one narrow question: can a generic latent state compress *triple-like* token interactions well enough to help on a standard language-model benchmark? This question is motivated by two observations. First, matrix-state and hybrid recurrent models suggest that richer state updates can recover more of the behavior usually attributed to attention [Mishra et al., 2026, Peng et al., 2024]. Second, mechanism-specific synthetic benchmarks, while useful, are not sufficient evidence that an architecture helps on generic sequence modeling.

39th Conference on Neural Information Processing Systems (NeurIPS 2025).

We therefore focus on a deliberately simple benchmark stack: byte-level WikiText-2 language modeling [Merity et al., 2017], a synthetic associative-recall stress test, and sequence-length throughput profiling on Apple MPS. Our model family is intentionally generic. It receives only token IDs, does not parse typed roles, and uses no task-specific symbolic heads.

The paper now reports two linked stories. First, the base triple-latent family already improves generic byte-level language modeling over a small Transformer. Second, the original latent-only models fail on associative recall, and the best fix is not to force exact retrieval into the same compressed state. Instead, a recall-focused hybrid adds a separate key-value path with gated late fusion.

Contributions.

- We define a generic triple-latent architecture family that combines a running token state with a compressed pair-memory pathway.
- We extend this family with a recall-focused hybrid that keeps the triple-latent path for compression while adding separate key-value memory, previous-token queries, and gated late fusion for retrieval.
- We provide a dedicated architecture figure and a standalone benchmark driver in the repository.
- We show that all three matched-width triple-latent variants improve small-scale byte-level WikiText-2 over a Transformer baseline, and that the strongest `triple-hybrid` variant still improves after approximate parameter matching.
- We show that the latent-only family fails on associative recall, but a gated hybrid retrieval path can outperform the Transformer on this benchmark on average across three seeds, albeit with substantial seed sensitivity.
- We show the corresponding systems caveat clearly: none of these models achieve a wall-clock throughput win in the current reference implementation.

2 Related Work

The first backdrop is the growing literature on efficient alternatives to quadratic attention. Depth-recurrent architectures such as Universal Transformers [Dehghani et al., 2019], linear-attention formulations that recover recurrent updates [Katharopoulos et al., 2020], retention and state-space replacements such as RetNet and Mamba [Sun et al., 2023, Gu and Dao, 2023], and recent recurrent revivals such as Gated DeltaNet, xLSTM, and matrix-state language models [Zhu et al., 2026, Beck et al., 2024, Mishra et al., 2026] all study how much of attention's behavior can be recovered with compressed state and linear-time sequence interaction. Our base triple-latent family belongs to this line, but explicitly separates a running token state from a compressed pair-memory path targeted at higher-order interactions.

The second backdrop is explicit memory and retrieval augmentation. Product-key memory, nearest-neighbor language models, and Memorizing Transformers show that exact or approximate associative lookup can complement parametric sequence modeling [Lample et al., 2019, Khandelwal et al., 2019, Wu et al., 2022]. Our recall-focused hybrid is closest in spirit to this family and to recent hybrid associative memory formulations [Lufkin et al., 2026], but it is deliberately minimal: retrieval is added only at the output layer, and the lossy recurrent compression path is left untouched.

Finally, recent production-oriented hybrids often combine multiple sequence mechanisms rather than committing to a single primitive. Compressive Transformers, Jamba, Samba, and Hymba illustrate different ways to mix local compression, attention, and recurrent or state-space components [Rae et al., 2019, Lieber et al., 2024, Ren et al., 2024, Dong et al., 2024]. Our goal here is smaller-scale and more diagnostic: use a controlled benchmark stack to separate what latent compression helps from what exact retrieval helps in a single generic architecture.

3 Method

3.1 Generic Triple-Latent Layer

Each layer takes token features $x_t \in \mathbb{R}^d$ and produces four sets of projections:

$$a_t, b_t, q_t^\ell, q_t^r \in \mathbb{R}^{H \times S}, \tag{1}$$

along with learned decay gates for a running state s_t and a compressed pair-memory pathway. The state update is

$$s_t = \lambda_t \odot s_{t-1} + (1 - \lambda_t) \odot a_t, \tag{2}$$

where λ_t is a learned per-token retention gate. Dense pair memory then stores an interaction between the *previous* state and the current write vector:

$$P_t = \gamma_t \odot P_{t-1} + (1 - \gamma_t) \odot (s_{t-1} \otimes b_t), \tag{3}$$

with $P_t \in \mathbb{R}^{H \times S \times S}$. Readout first probes the compressed pair memory with a left query and then composes the result with a right query:

$$o_t = W_o \operatorname{vec}\big((P_t q_t^\ell) \odot q_t^r\big)\,. \tag{4}$$

The layer output adds this mixed readout back to the residual stream and then applies an FFN block.

3.2 Variants

We study three generic variants.

- **Triple-Latent**: dense pair memory P_t.
- **Triple-Slot**: replace dense pair memory with learned slots that store low-rank left and right factors.
- **Triple-Hybrid**: keep dense pair memory and add a local convolutional path before the recurrent stack.

Unlike benchmark-aware typed latent models, these variants do not expose special roles, tags, or symbolic task heads.

3.3 Gated Hybrid Associative Retrieval

The base triple-latent family compresses context well, but it does not provide explicit content-addressable lookup. To test whether exact retrieval should be handled separately, we add a recall-focused hybrid that keeps the triple-latent stack and augments only the top layer with an explicit key-value memory.

Given token features e_t, the retrieval path writes key-value pairs (k_{t-1}, v_t) into an append-only memory using learned projections

$$k_t = W_k e_t, \qquad v_t = W_v e_t. \tag{5}$$

At retrieval time, we query the memory using the *previous-token* representation,

$$q_t = W_q e_{t-1}, \tag{6}$$

and compute top-k similarity scores against stored normalized keys:

$$\alpha_{t,i} \propto \exp\left(\frac{\hat{q}_t^\top \hat{k}_i}{\sqrt{d_a}}\right). \tag{7}$$

The retrieved memory vector is

$$m_t = \sum_i \alpha_{t,i} v_i. \tag{8}$$

We then apply a learned read gate and fuse only at the output layer:

$$\tilde{m}_t = \sigma(W_g q_t) \odot m_t, \qquad \ell_t = W_{\text{vocab}} h_t + W_{\text{vocab}}(\mathrm{LN}(\tilde{m}_t))\,. \tag{9}$$

This design leaves the triple-latent state untouched by retrieval writes and keeps exact lookup separate from lossy recurrent compression.

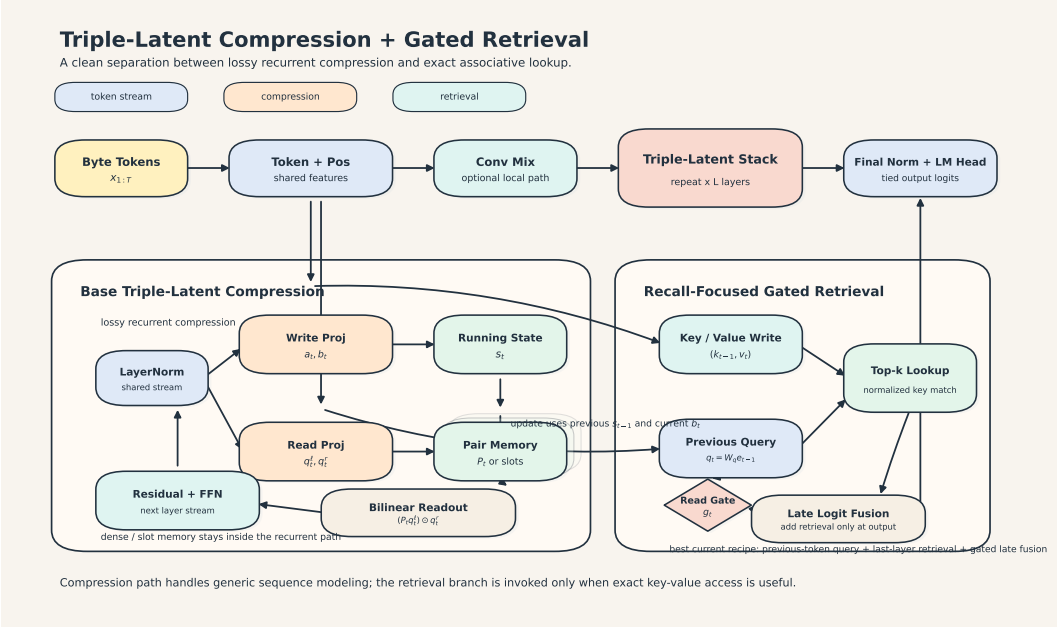

Figure 1: Standalone overview of the generic triple-latent family. Tokens are embedded, optionally mixed by a local convolution, and passed through a stack of recurrent layers that maintain a running state and a compressed pair-memory pathway. The dense, slot, and hybrid variants differ only in how the pair-memory path is represented and whether local convolution is added. The recall-focused extension in this paper adds a separate explicit key-value retrieval branch on top of this base stack.

4 Experimental Setup

Language modeling. We train byte-level autoregressive models on WikiText-2 raw text [Merity et al., 2017]. Inputs are UTF-8 bytes with a 259-token vocabulary. The matched-width setting uses $d = 64$, 3 layers, 4 heads, sequence length 128, batch size 16, and 80 optimizer steps on Apple MPS.

Associative recall. We also train each model on a synthetic key-value retrieval task with four key-value pairs, random filler, and a final query. Models run for 200 steps with batch size 32 and sequence length 128. Accuracy is measured on the final answer token only.

Recall-focused hybrid follow-up. Because the original latent-only family and early no-gate dual-memory variants remained near chance on associative recall, we run a second memory study for the gated hybrid described above. We sweep widths $d \in \{64, 80, 96\}$ at 500 associative-recall training steps, then run three seeds $(7, 19, 29)$ for the best width against same-width Transformer and plain `triple-hybrid` baselines. We also report byte-level WikiText-2 and throughput for the best recall-focused configuration under the same 80-step LM schedule used elsewhere in the paper.

Stabilization follow-up. We also test whether the gated winner can be made more reliable with simple optimization changes alone. This follow-up keeps the same architecture and uses a 50-step learning-rate warmup, a sequence-length curriculum from 32 to 128 over the first 150 steps, and read-gate bias initialization at -2.0. We again report a three-seed mean at $d = 64$.

Parameter-matched follow-up. To check whether the WikiText-2 gain is only a width effect, we run an approximate parameter-matched language-model follow-up using the same optimizer schedule but reduced widths: `triple-latent` and `triple-slot` use $d = 60$, and `triple-hybrid` uses $d = 56$. These settings place all three variants near the Transformer's 174,848 parameters.

Regime	Model	Params	Val. BPB ↓	Val. PPL ↓	Recall ↑
Matched width	Transformer	174,848	5.124	34.88	**25.4%**
Matched width	Triple-Latent	187,928	5.022	32.50	13.4%
Matched width	Triple-Slot	188,024	5.041	32.92	13.4%
Matched width	Triple-Hybrid	208,472	**4.766**	**27.21**	13.4%
Parameter-matched LM	Triple-Latent	170,424	5.114	34.64	–
Parameter-matched LM	Triple-Slot	170,520	5.126	34.92	–
Parameter-matched LM	Triple-Hybrid	**169,424**	**5.067**	**33.52**	–
Budget winner	Triple-Hybrid $(64, 8)$	177,752	**4.806**	**27.98**	13.4%

Table 1: Generic triple-latent results. All three matched-width variants improve over the Transformer baseline on byte-level WikiText-2. The strongest `triple-hybrid` variant still improves after approximate parameter matching, and a targeted near-budget sweep yields an even stronger three-seed winner. None of the *latent-only* variants improve associative recall.

Throughput benchmark. We measure forward-only inference on Apple MPS at sequence lengths $\{64, 128, 256, 512\}$ with batch size 8 and 20 timed iterations. The Transformer benefits from optimized attention kernels, while the triple-latent models remain Python-loop reference implementations.

Compute accounting. The main training comparisons in the paper are still step-matched rather than FLOP-matched. To quantify the arithmetic profile more clearly, we also estimate dominant forward FLOPs analytically from the implemented model definitions. The estimate counts the main matrix multiplies and tensor contractions, the `triple-hybrid` local convolution, and any extra vocabulary projection from late logit fusion. It omits embedding lookups, normalization, softmax, top-k selection, and Python overhead.

5 Results

5.1 Generic Benchmark

Table 1 is the main result for the *base* triple-latent family. In the matched-width setting, all three triple-latent variants beat the Transformer on byte-level WikiText-2. The strongest `triple-hybrid` variant reaches 4.766 bits-per-byte versus 5.124 for the Transformer baseline. This improvement is not entirely explained by parameter count: in the approximate parameter-matched follow-up, `triple-hybrid` still reaches 5.067 bits-per-byte at fewer parameters than the Transformer.

We then pushed the search further under a near-baseline parameter budget. A targeted sweep over `triple-hybrid` and `triple-slot-hybrid` variants found the strongest fair-budget configuration at `triple-hybrid` with $d = 64$ and state dimension 8. That model uses 177,752 parameters, reaches 4.827 bits-per-byte in the pilot run, and achieves 4.806 mean bits-per-byte across three seeds. The corresponding Transformer baseline averages 5.143 across the same three seeds. This makes the generic result stronger than the earlier single-run comparison alone: the quality win survives both a targeted budget sweep and a small robustness check.

5.2 Retrieval Ablation

Before adding gating, we tested whether the recall failure was simply a query, fusion, or layer-placement problem. Table 2 shows that the answer is no. Parallel, serial, and previous-token queries all remain at 13.4% accuracy in the short dual-memory sweeps. Last-layer-only retrieval helps efficiency relative to all-layer retrieval, and logit fusion helps byte- level language modeling much more than sum or concat fusion, but these changes alone do not move associative recall.

The critical transition is therefore not "use previous-token query" by itself. It is "use previous-token query, read only at the last layer, fuse at the logits, and add a learned read gate." A same-budget long-run check of an ungated previous-query model also stays near the old failure mode at 11.96% accuracy, which reinforces this conclusion.

Variant	Query	Layers	Fusion	Gate	Recall ↑	Val. BPB ↓
Transformer	–	–	–	–	25.4%	5.144
Triple-Hybrid	–	–	–	–	13.4%	4.843
Assoc sum	par.	all	sum	no	13.4%	4.835
Assoc concat	par.	all	concat	no	13.4%	4.767
Assoc logits	par.	all	logits	no	13.4%	3.664
Serial sum	serial	all	sum	no	13.4%	4.834
Prev. logits	prev.	all	logits	no	13.4%	3.657
Last logits	par.	last	logits	no	13.4%	3.680
Gated winner	prev.	last	logits	yes	**41.9%**	**3.595**

Table 2: Retrieval ablation for the recall-focused hybrid family at $d = 64$. Ungated rows come from the 200-step dual-memory sweeps, while the final row reports the 500-step three-seed mean of the gated winner. Query and fusion choices affect LM compression much more than recall; the first configuration that changes recall materially is previous-token query plus last-layer retrieval plus a read gate and late logit fusion.

Model	Recall mean ↑	Best seed ↑	Val. BPB ↓	Tok/s 64 ↑	Tok/s 512 ↑
Transformer	25.0%	25.6%	5.144	602,684	344,185
Triple-Hybrid	11.6%	12.0%	4.843	18,433	18,870
Gated Hybrid Retrieval	**41.9%**	**100.0%**	**3.595**	5,659	4,910

Table 3: Recall-focused gated hybrid at $d = 64$. Recall numbers are three-seed means over 500 associative-recall steps, with the best single-seed run shown separately. Byte-level WikiText-2 and throughput use the same 80-step LM setup as the rest of the paper. The gated hybrid improves both recall and LM compression, but remains slower than the plain triple-hybrid and much slower than the Transformer.

5.3 Recall-Focused Hybrid

The ablation above narrows the search space but still leaves a clear gap: compression improves, but exact retrieval does not. We therefore ran a second, recall-focused follow-up on the previous-query, last-layer, logit-fusion path with an added read gate. This is the first configuration that changes the benchmark meaningfully.

Table 3 summarizes the best recall-focused model at $d = 64$. Across three seeds, `triple-hybrid-assoc-gated-last-logits` reaches 41.9% mean associative-recall accuracy, compared with 25.0% for the Transformer and 11.6% for plain `triple-hybrid`. The best seed solves the task at 100.0% accuracy. Just as importantly, longer training alone does not explain the gain: same-budget ungated baselines remain near 12%.

The result is encouraging but not yet fully stable. The recall gain is highly seed-sensitive: one seed converges to exact retrieval, while two remain near the old failure mode. Width scaling is also non-monotonic in the current implementation: a single-seed sweep gives 100.0% at $d = 64$, 67.2% at $d = 80$, and a collapse back to 11.96% at $d = 96$. We therefore treat the gated hybrid as a promising direction rather than a solved replacement.

A later optimization-only stabilization attempt strengthens that caution rather than weakening it. The warmup-plus-curriculum retraining described above reaches only 12.7% mean recall across the same three seeds, with a best seed of 13.4%. In other words, the original high-recall mode is not reproduced by simple schedule shaping or gate-bias initialization alone.

5.4 Approximate Compute Accounting

The training runs in this paper are still step-matched, not FLOP-matched, but an analytic forward estimate is still informative. Table 4 counts the dominant tensor contractions from the implemented models. At the training context of 128, plain `triple-hybrid` uses about $0.80\times$ the Transformer's dominant forward FLOPs, and the gated hybrid uses about $0.95\times$. At sequence length 512, these estimates fall to $0.47\times$ and $0.60\times$, respectively.

Model	Seq 64	Seq 128	Seq 512
Transformer	24.1 (1.00x)	54.6 (1.00x)	369.3 (1.00x)
Triple-Hybrid	21.8 (0.90x)	43.5 (0.80x)	174.0 (0.47x)
Gated Hybrid Retrieval	25.8 (1.07x)	52.1 (0.95x)	220.9 (0.60x)

Table 4: Approximate forward MFLOPs from the dominant tensor contractions in the repo implementation. Counts include the hybrid local convolution and the gated model's extra logits head, but omit embedding lookups, normalization, softmax, top-k selection, and Python overhead.

The wall-clock result is the opposite because arithmetic count is not the whole story in the current codebase. The Transformer runs through optimized attention kernels, while the triple-latent models execute recurrent Python loops and unfused retrieval logic. The gated model also reintroduces quadratic sequence-length cost through associative scoring, even though its constant is smaller than full attention in this small configuration.

5.5 Memory and Throughput Tradeoffs

The expanded results now come with two clear caveats.

First, explicit retrieval is no longer a simple failure story, but it is still not cleanly solved. The latent-only variants remain at 13.4% after the original short runs, and the best gated hybrid is strongly seed-sensitive even though its three-seed mean surpasses the Transformer. A later warmup-plus-curriculum retraining attempt falls back to 12.7% mean recall, so there is still no simple recipe that stabilizes the high-recall mode.

Second, the current implementation is still not a systems win. The plain `triple-hybrid` is about $32.7\times$ slower than the Transformer at sequence length 64 and $18.2\times$ slower at length 512. The recall- focused gated hybrid is about $106.5\times$ and $70.1\times$ slower at the same lengths, despite the more favorable arithmetic profile in Table 4. The recurrent path plus explicit retrieval are both still unfused Python reference code.

6 Discussion

The central lesson is that generic latent compression appears to be real, but exact retrieval should not be forced into the same compressed state. The positive language-model result is not limited to a benchmark-aware symbolic construction: a fully untyped latent family improves a standard language-model benchmark, and the strongest variant survives approximate parameter matching. That makes the compression story more credible than a synthetic benchmark win alone.

The memory result sharpens the picture further. A separate key-value path with late gated fusion can recover associative recall and even outperform the Transformer on average in our three-seed $d = 64$ follow-up. But the behavior is still unstable across seeds and widths. A later warmup-plus-curriculum retraining attempt also fails to reproduce the original high-recall mode, which suggests that the remaining problem is not merely a missing warmup schedule. This suggests that the next research steps should focus on two fronts:

- better optimization and routing for the gated retrieval path, so the high-recall solution becomes a stable mode rather than an occasional one, and so it transfers beyond the dedicated associative-recall benchmark
- fused scan, fused retrieval, and custom recurrent kernels, so the architecture can realize its favorable arithmetic profile in wall-clock terms

7 Limitations

This is a pilot study with several important limits.

- Most experiments are small Apple MPS runs. The key recall result uses three seeds, but the broader sweep is still far from exhaustive.

- We compare matched training steps, not matched training FLOP budgets. Table 4 is an analytic forward estimate, not a full training compute accounting.
- An exploratory zero-shot RULER-core check remained at 0.0 at context lengths 128 and 256 for the small byte-level checkpoints used here. We therefore do not treat RULER as a main result in this paper, and we do not yet report broader long-context suites such as LongBench, full RULER, BABILong, and NoLiMa [Bai et al., 2023, Hsieh et al., 2024, Kuratov et al., 2024, Modarressi et al., 2025].
- The gated associative-recall improvement is seed-sensitive, with one seed solving the task and others remaining much weaker. Our optimization-only stabilization follow-up does not recover that strong mode.
- The throughput numbers come from a Python-loop reference implementation, not an optimized recurrent kernel.

8 Conclusion

Generic triple-latent compression is strong enough to merit its own line of investigation, but the updated story is now two-part. First, the base triple-latent family improves generic byte-level language modeling over a small Transformer baseline, and the best `triple-hybrid` remains competitive under approximate parameter matching. Second, exact retrieval becomes substantially stronger when we add a separate gated key-value path on top of the triple-latent stack: the best recall-focused hybrid beats the Transformer on mean associative recall in our three-seed $d = 64$ follow-up and reaches 100.0% in its best run. The result is still unstable across seeds and much slower than attention. An optimization-only stabilization attempt also fails to reproduce the original high-recall mode. So this is not yet a practical replacement for standard attention. But it is meaningful evidence that latent compression and exact retrieval can coexist in a single generic architecture when they are routed through separate pathways.

A Implementation Notes for a Competitive Kernel

The current codebase is a research reference implementation, not a production kernel stack. A serious systems comparison against modern attention would need at least four engineering upgrades.

- **Fused recurrent scan.** The state update, pair-memory update, and readout should run as a single fused scan across sequence positions rather than a Python loop over tokens.
- **On-device associative lookup.** The explicit key-value path needs a tiled similarity kernel, top-k selection, and value aggregation on device. The current list-append plus stack formulation is simple but bandwidth-inefficient.
- **Late-logit fusion without duplicate overhead.** The gated model currently pays for an additional vocabulary projection. A stronger systems implementation would either fuse the two projections or share more of the final readout path.
- **Decode-time cache layout.** Autoregressive decoding needs a persistent cache for the recurrent state, pair-memory tensor, and the explicit key-value memory. These should be laid out separately so recurrent updates stay contiguous while retrieval remains append-only.
- **Compiled backward path.** A training-speed comparison also needs fused backward kernels or at least aggressive graph compilation. Without this, the measured wall-clock gap says more about implementation maturity than about the architecture's arithmetic shape.

These requirements explain why the current throughput numbers are much worse than the FLOP estimates alone would suggest: the baseline Transformer already benefits from optimized attention kernels, while the triple-latent models do not yet benefit from equivalent recurrent or retrieval kernels.

References

Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. Longbench: A bilingual, multitask benchmark for long context understanding, 2023.

Maximilian Beck, Korbinian Pöppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. xLSTM: Extended long short-term memory, 2024.

Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. In *International Conference on Learning Representations*, 2019.

Xin Dong, Yonggan Fu, Shizhe Diao, Wonmin Byeon, Zijia Chen, Ameya Sunil Mahabaleshwarkar, Shih-Yang Liu, Matthijs Van Keirsbilck, Min-Hung Chen, Yoshi Suhara, Yingyan Lin, Jan Kautz, and Pavlo Molchanov. Hymba: A hybrid-head architecture for small language models, 2024.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2023.

Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What's the real context size of your long-context language models?, 2024.

Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *International Conference on Machine Learning*, 2020.

Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models, 2019.

Yuri Kuratov, Aydar Bulatov, Petr Anokhin, Ivan Rodkin, Dmitry Sorokin, Artyom Sorokin, and Mikhail Burtsev. BABILong: Testing the limits of llms with long context reasoning-in-a-haystack, 2024.

Guillaume Lample, Alexandre Sablayrolles, Marc'Aurelio Ranzato, Ludovic Denoyer, and Hervé Jégou. Large memory layers with product keys, 2019.

Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, Erez Safahi, Shaked Meirom, Yonatan Belinkov, Shai Shalev-Shwartz, Omri Abend, Raz Alon, Tomer Asida, Amir Bergman, Roman Glozman, Michael Gokhman, Avashalom Manevich, Nir Ratner, Noam Rozen, Erez Shwartz, Mor Zusman, and Yoav Shoham. Jamba: A hybrid transformer-Mamba language model, 2024.

Leon Lufkin, Tomás Figliolia, Beren Millidge, and Kamesh Krishnamurthy. Hybrid associative memories, 2026.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *International Conference on Learning Representations*, 2017.

Mayank Mishra et al. Non-linear RNNs with matrix-valued states for scalable language modeling, 2026.

Ali Modarressi, Hanieh Deilamsalehy, Franck Dernoncourt, Trung Bui, Ryan A. Rossi, Seunghyun Yoon, and Hinrich Schütze. Nolima: Long-context evaluation beyond literal matching, 2025.

Bo Peng, Tom Goldstein, et al. Eagle and finch: RWKV with matrix-valued states and dynamic recurrence, 2024.

Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, Felix Hill, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling, 2019.

Liliang Ren, Yang Liu, Yadong Lu, Yelong Shen, Chen Liang, and Weizhu Chen. Samba: Simple hybrid state space models for efficient unlimited context language modeling, 2024.

Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Fei Xia, Shuang Wang, and Furu Wei. Retentive network: A successor to transformer for large language models, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

Yuhuai Wu, Markus N. Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers, 2022.

Di Zhu et al. Gated DeltaNet: A sequential, linear-time architecture for language modeling, 2026.